ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

Compte rendu de TP

Cadrer une application IA et préparer l'environnement

Développement et déploiement d'applications intelligentes

Réalisé par
Othmane Moussawi
en binôme avec Ahmed Rayane Ramzi

Enseignant : A. Larhlimi

Année scolaire : 2026–2027
14 octobre 2026

Table des matières

1 Introduction **1**

2 Cadrage de trois cas **1**
2.1 Démarche 1
2.2 Réalisation 1
2.3 Interprétation 1

3 Environnement et dépôt Git **2**
3.1 Démarche 2
3.2 Réalisation 2
3.3 Interprétation 2

4 Incidents rencontrés **2**

1. Introduction

Ce TP pose les fondations du projet `sentiment-app`. Il comporte deux parties : cadrer trois besoins métier avant tout développement, puis monter un dépôt Git reproductible, testé et partageable. Les objectifs sont de choisir l'approche adaptée à chaque cas, d'installer un environnement Python isolé et de faire passer les contrôles de diagnostic.

2. Cadrage de trois cas

Énoncé

Pour chacun des trois cas (ordonnances incomplètes en pharmacie, questions sur le règlement intérieur, avis clients négatifs), choisir une approche parmi : règles métier, machine learning, RAG ou modèle génératif seul. La justification doit citer au moins deux critères parmi (a) données étiquetées, (b) vérifiabilité, (c) coût par requête, (d) conséquence d'une erreur.

2.1 Démarche

L'arbre de décision se lit dans l'ordre. Si la logique est connue et stable, on retient les **règles métier**. Sinon, si l'on dispose de données étiquetées, on retient le **machine learning**. Si la réponse doit être tirée de documents et citée, on retient le **RAG**. Le modèle génératif seul n'est retenu qu'en dernier recours.

2.2 Réalisation

Chaque fiche de cadrage est rédigée dans `docs/cadrage/` (`cas1-pharmacie.md`, `cas2-reglement.md`, `cas3-avis-negatifs.md`). Les rubriques obligatoires sont remplies et une seule case est cochée par fiche.

Cas	Approche	Critères principaux
Ordonnances incomplètes	Règles métier	(a) pas d'étiquettes ; (b) explicabilité ; (d) erreur grave
Règlement intérieur	RAG	(b) citer l'article ; (a) pas d'étiquettes ; (c) volume faible
Avis négatifs	Machine learning	(a) étiquettes gratuites ; (c) volume élevé ; (d) erreur peu grave

Table 1 – Approche retenue pour chaque cas.

2.3 Interprétation

Pour le cas 3, le machine learning s'impose : la note en étoiles fournit gratuitement l'étiquette, le volume justifie un coût par requête très faible, et une erreur n'a qu'une conséquence limitée. Les deux autres cas montrent pourquoi l'approche change : le cas 1 relève de règles, car la liste des mentions obligatoires est réglementaire et une erreur est grave ; le cas 2 relève du RAG, car la réponse doit citer l'article source.

Réponse

Cas 1 : règles métier. Cas 2 : RAG. Cas 3 : machine learning.

3. Environnement et dépôt Git

3.1 Démarche

Le kit `sentiment-app` est décompressé, puis un environnement virtuel est créé et les dépendances installées. Le diagnostic `scripts/check_setup.py` et les tests `pytest` vérifient la structure, les paquets et le `.gitignore`. Enfin, le dépôt Git est initialisé, versionné et publié sur GitHub.

3.2 Réalisation

— Environnement virtuel créé avec `py -3.12 -m venv .venv`. Le poste ne disposant pas de Python 3.11, la version 3.12 a été utilisée ; le diagnostic l'accepte avec un avertissement, et la CI tourne en 3.11.

— Paquets installés : `pandas`, `scikit-learn`, `pytest`, puis le reste de `requirements.txt` pour la suite.

— Tests de cadrage (`tests/test_cadrage.py`) : 12 tests verts après rédaction des fiches.

— Diagnostic `check_setup.py` : **0 erreur**, 2 avertissements dont un seul auteur de commits, avant que le binôme ne pousse son propre commit.

— Git : `.venv/` n'apparaît pas dans `git status`, le `.gitignore` l'exclut bien.

Résultat de `pytest` sur le dépôt avant le commit du second membre : **44 passed, 1 failed**. Le seul échec est `test_each_member_has_committed`, qui exige deux auteurs de commits. Ce résultat ne correspond pas aux valeurs indiquées par l'énoncé (3 failed, 31 passed, 4 skipped) : le dépôt ayant déjà un historique et les fiches étant remplies, les tests conditionnels ne sont plus ignorés. Nous n'avons pas poussé plus loin cette comparaison.

3.3 Interprétation

Le point bloquant de cette partie est l'historique Git : un test vérifie qu'au moins deux personnes ont commit, ce qui ne peut être garanti que si chaque membre pousse depuis son propre poste et son propre compte. Le fichier `requirements.lock` est généré par `setup.ps1` sur chaque poste : un seul membre le commit.

Réponse

Environnement en place, diagnostic sans erreur, 44 tests verts sur 45. Le 45e test attend le commit du second membre.

4. Incidents rencontrés

— **Environnement virtuel déplacé.** Le `.venv` pointait vers un ancien emplacement et était vide. Il a été supprimé puis recréé : un environnement virtuel n'est pas déplaçable.

— **Fiches de cadrage hors du dépôt.** Les premières fiches avaient été rédigées hors de `docs/cadrage/`. Les fichiers versionnés restaient vides, et `test_cadrage.py` échouait (3 échecs). Corrigé en réécrivant les fiches dans le bon dossier.

— **Un seul auteur de commits.** Les commits initiaux portaient le même `user.name` ; le test d'historique reste rouge jusqu'au commit du second membre.

Conclusion

Les trois cas ont reçu une approche justifiée par les critères de la grille, et l'environnement répond aux contrôles de diagnostic. Le dépôt est prêt à recevoir la suite du projet dès que le binôme a poussé son commit.